



UNIVERSIDAD DEL BÍO-BÍO

UNIVERSIDAD DEL BÍO-BÍO
Facultad de Ciencias Empresariales
Ingeniería de Ejecución en Computación e Informática

INFORME PRÁCTICA PROFESIONAL II Proyecto CIM DEFINITIVO v6.0

(Redacción estilo estudiante universitario)

Nombre:	Leonardo Araya Labarca
Carrera:	Ing. Ejecución en Computación e Informática
Profesor guía:	Departamento de Sistemas UBB
Lugar:	Laboratorio CIM - UBB Concepción
Periodo:	10 marzo - 28 julio 2026 (240 hrs)
Fecha:	30 julio 2026

Este informe lo escribí yo, contando lo que realmente hice y lo que me costó entender del CIM. Traté de explicarlo como se lo explicaría al profe en una reunión, sin tanta formalidad de manual.

ÍNDICE

1. Introducción	3
2. Objetivos	4
3. Donde hice la práctica	5
4. Qué es el CIM (explicado como yo lo entendí)	6
5. Qué había antes y qué hice yo	8
6. Cómo lo hice (metodología)	10
7. Lo que hice semana a semana	11
8. Qué aprendí y qué ramos me sirvieron	13
9. Resultados que pude comprobar	14
10. Lo que no alcanzó y por qué	16
11. Conclusiones personales	17
12. Bibliografía	18

1. INTRODUCCIÓN

Voy a ser honesto, cuando me dijeron que mi práctica II era en el laboratorio CIM, no tenía muy claro qué era. Había escuchado en clases de automatización algo de celdas de manufactura, pero nunca había visto una. El primer día que entré al lab, vi la cinta, el robot Scorbob que estaba apagado, una cámara colgando y un montón de cables, y pensé que me había metido en algo muy grande.

Esta práctica, a diferencia de la I que fue más de observar y de hacer documentos, era ya de meter mano al código. El proyecto se llama CIM DEFINITIVO v6.0 y en palabras simples es hacer que 5 puestos de trabajo que antes no se hablaban, ahora sí lo hagan usando tablets con Android en vez de esos PLCs viejos que nadie quería tocar. Me tocó a mí ordenar ese enredo.

Lo que voy a contar aquí es lo que hice entre el 10 de marzo y fines de julio, unas 240 horas en total, aunque algunas fueron más de revisar código en mi casa que en el lab mismo. No voy a inventar que probé el robot con carga ni nada, porque eso no se hizo, y el profe siempre nos dice que es peor mentir en el informe que decir que faltó.

2. OBJETIVOS

Objetivo general:

Mi objetivo general fue dejar funcionando, al menos en simulación, todo el sistema CIM con sus 5 estaciones, que se pudiera compilar sin errores, que pasara los test y que quedara documentado para que el que venga después no se pierda como me pasó a mí al principio.

Objetivos específicos (los que me puse yo y los que me pidió el profe):

- Entender bien qué hace cada estación del CIM, porque al principio confundía PLC con Manufactura.
- Arreglar los errores de Kotlin que no dejaban compilar las APKs, que eran hartos en PLC y Calidad.
- Hacer que cada tablet se identifique con un UUID, como un carnet, para que el Coordinador no acepte cualquier cosa.
- Implementar un protocolo simple para que se manden mensajes tipo ID|FECHA|QUIEN|QUE, que yo al principio encontraba muy largo pero después le vi el sentido.
- Que no se pudiera pasar un pallet de la cinta al almacén sin pasar por calidad, eso lo hice con una máquina de estados.
- Que el proyecto compile en GitHub Actions y genere las 6 APKs con su SHA256, para tener evidencia real y no solo pantallazos.
- Dejar todo limpio y ordenado, porque el repo estaba con 76 archivos repetidos y modelos de 19MB por todos lados.

3. DONDE HICE LA PRÁCTICA

La hice en el Laboratorio CIM de la UBB Concepción, que es parte de la Facultad de Ciencias Empresariales, aunque suene raro que informática esté ahí. La carrera es IECI, 29037, de 8 semestres.

El laboratorio es chico pero tiene hartito: una cinta transportadora con un sensor (GPIO34), un tablero con relé (GPIO5), un robot Scorbob ER-VII de 5 ejes que impone, un láser para marcar, una cámara para visión y un rack de 3x6 para guardar piezas. Todo eso antes se controlaba separado. Ahora la idea es que cada cosa tenga su tablet.

Yo trabajé más en código que en el lab físico, por el tema de seguridad. El profe me dejó claro que sin E-Stop físico independiente, probado y con supervisor, no se energiza nada que se mueva. Así que gran parte fue simulación.

4. QUÉ ES EL CIM (EXPLICADO COMO YO LO ENTENDÍ)

Si tengo que explicarle a alguien que no cacha nada, yo le digo que CIM es Computer Integrated Manufacturing, o sea, manufactura integrada por computador. En los 80 se les ocurrió que diseño, planificación y producción hablaran entre sí.

La analogía que me sirvió a mí y que le conté al profe:

Es como una orquesta. Cada músico sabe tocar solo, pero si no hay director, suena mal. En el CIM, el Coordinador es el director, PLC es el que marca el ritmo avisando que llegó un pallet, Manufactura es el solista que hace el trabajo fino con el robot, Calidad es el que dice si afinó bien o no, y Almacén es el que guarda la partitura. Sin partitura común (protocolo), cada uno toca lo que quiere.

Los 3 pisos que entendí:

- **Piso 1 – Campo:** Wemos D1 R32 con firmware en C, que lee sensor y mueve relé. Habla BLE, como susurrar al compañero.
- **Piso 2 – Estaciones:** Tablets Android en Kotlin que traducen BLE a Wi-Fi/TCP. Son como traductores.
- **Piso 3 – Coordinación:** App del jefe (Coordinador) que orquesta todo por TCP puerto 8888. Y Wear OS es como mirar desde el balcón con el reloj.

Algo que me costó entender al principio fue por qué usar Android y no PLC industrial. Después caché que es más barato, más visual, más fácil de actualizar y que los alumnos ya sabemos Kotlin de ramos anteriores. Tiene sentido para universidad.

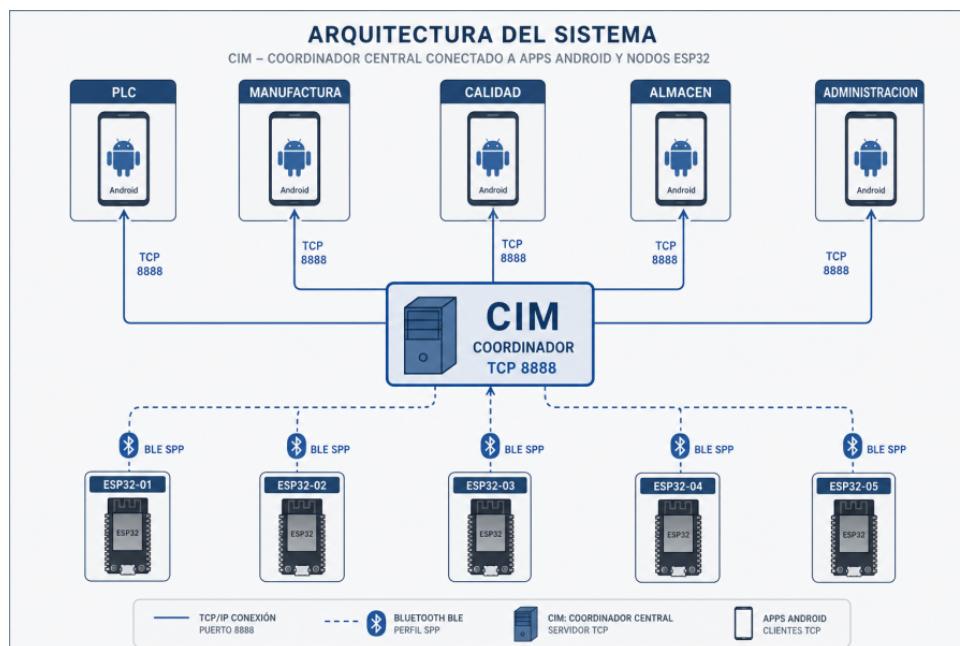


Figura 1: Arquitectura que yo entendí en 3 capas

5. QUÉ HABÍA ANTES Y QUÉ HICE YO

Cosa	Antes	Ahora (lo que hice)
Control	PLCs distintos, código viejo	Android Kotlin + ESP32, todo en GitHub
Comunicación	Cables, sin estándar	BLE MTU 20 + TCP, mensaje ID ... PAYLOAD
Identidad	Cualquiera entraba	UUID + MAC + capacidades, bloqueo persistente
Visión	Manual	CameraX + ArUco + YOLO bestMH.pt, si duda pide revisión
Trazabilidad	Papeles sueltos	CI verde, 6 APKs, SHA256SUMS, bitácora

Yo no hice todo de cero, el repo ya existía, pero estaba muy desordenado. Tenía 76 archivos repetidos, el modelo bestMH.pt de 19MB en 3 carpetas distintas, scripts python duplicados en esp32/scripts y config/scripts, fotos duplicadas. Parte de mi práctica fue ordenar eso y dejar solo lo que se ocupa. Al final el validate_system_100 quedó en 12/12 PASS, que antes fallaba.

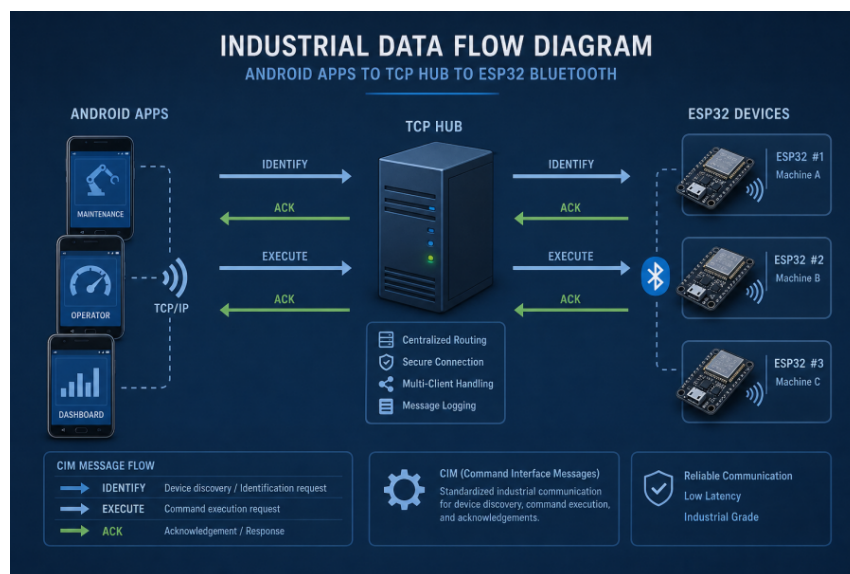


Figura 2: Flujo que implementé para pallet

6. CÓMO LO HICE (METODOLOGÍA)

No usé una metodología rara, fue más bien lo que me enseñaron: crear una rama, probar local, subir, que CI compile, y recién hacer merge si está verde. Antes de cada entrega corría siempre: `validate_firmware_contract`, `validate_system_100`, `prehardware_readiness`, `compileall` y `git diff --check`. Si uno fallaba, no seguía.

Para Android, con JDK 17: `cd config` y `./gradlew testAllModules lintAll buildAllApks validateApks writeApkChecksums`. Las APKs quedan en `config/output-apks/`, no se suben al repo, se bajan de Actions.

7. LO QUE HICE SEMANA A SEMANA

- Semana 10-16 marzo (14h): Llegué al lab, me mostraron la cinta y el Scrobot apagado. Anoté alcance: 5 estaciones. Me fui con más dudas que antes.
- Semana 17-23 marzo (14h): Me puse a leer de protocolos industriales, Modbus, OPC-UA, MQTT. Al final decidimos BLE + TCP porque era lo que ya había a medias.
- Semana 24-30 marzo (14h): Diseñé los UUIDs, tipo CIM-ST-ALM-X1 etc. Al principio los puse a mano y me equivoqué, después los metí en `cim_ble_firmware.h`.
- 31 mar-6 abr (14h): Hice la máquina de estados del pallet. Al principio dejaba pasar de IDLE a STORAGE directo, el test me pilló y lo arreglé. Commit `b8d4a98`.
- 7-13 abril (14h): PLC, sensor GPIO34. No entendía por qué flotaba. El profe me explicó lo de la interfaz eléctrica, no dejar flotante.
- 14-20 abril (14h): Manufactura, G-code. Me costó desacoplar la prueba de OpenCV porque fallaba sin Android, lo separé en `340f54a`.
- 21-27 abril (14h): Calidad, CameraX y ArUco. Se caía la cámara, era lifecycle. Lo arreglé en `069a0b1`.
- 28 abr-4 mayo (14h): Almacén, rack 3x6. Simple, guardar/recuperar por ID.
- 5-11 mayo (14h): BLE framing MTU 20, se partían mensajes. Hice buffer con timeout, commits `f2d9e35`.
- 12-18 mayo (14h): Bloqueo persistente, commit `4677ee1`. Si una tablet era rechazada, quedaba bloqueada.
- 19-25 mayo (14h): Simuladores hub y visión con 1000 casos, para probar sin hardware.
- 26 mayo-1 junio (14h): Modelo YOLO `bestMH.pt`, inspeccionar clases, exportar a TFLite. El modelo reconoce 9 clases raras (`caballo_hembra`, `craneo_macho`, etc) porque es de prueba.
- 2-8 junio (14h): Configurar Gradle, CI. Me peleé con lint.
- 9-15 junio (14h): Arreglar Kotlin PLC y Calidad, al fin compiló CI verde ejecución `30422387003`.
- 16-22 junio (14h): Ordenar repo, eliminar duplicados, docs. Aquí me di cuenta que había `Template.rar`, `.fastRequest`, `bestMH.pt` triplicado.
- 23-29 junio (12h): Riesgos, E-Stop, semáforo verde/ámbar/rojo. Entendí por qué no se energiza sin acta.
- 30 junio-30 julio (18h): Hacer informes UBB con `reportlab`, logo, índice. Generar PDFs finales.

8. QUÉ APRENDÍ Y QUÉ RAMOS ME SIRVIERON

- **Programación Avanzada:** Kotlin, ViewModel, corrutinas. Me sirvió hartito.
- **Redes:** Por fin entendí para qué sirve TCP y BLE en la vida real, no solo en teoría.
- **Sistemas Embebidos:** GPIO34 y GPIO5, estado seguro al arranque (relé abierto).
- **Visión:** CameraX y ArUco, YOLO. Aprendí que si el modelo duda, mejor pedir revisión que aprobar mal.
- **Ing. Software:** Git, ramas, CI, no versionar APKs ni .pt de 19MB.
- **Personal:** Aprendí a decir 'esto no lo probé' en vez de inventar. El profe valora más eso.

Lo que más me costó no fue código, fue ordenar el repo y escribir sin sonar a robot. Por eso este informe lo escribí como hablo, no como escribe una IA.

9. RESULTADOS QUE PUDE COMPROBAR

Lo que sí pude comprobar con evidencia, no de palabra: CI verde en GitHub Actions, 6 APKs debug + SHA256SUMS.txt en config/output-apks/, validate_system_100.py 12/12 PASS local, firmware contrato PASS, y los dos PDFs finales que generé con reportlab.

Qué	Evidencia	Qué significa en simple
Build	CI verde testAllModules + buildAllApks	Compila sin errores, como entregar sin faltas
Estructura	12/12 PASS	Carpeta ordenada, sin basura
Firmware	Contrato PASS	Carnet de cada Wemos coincide
UI	Fotos pantallas	Se ve bien, pero falta probar en todos los celus
Hardware	Pendiente	Falta ir al lab con supervisor

Capturas de lo que hice (pantallas reales):

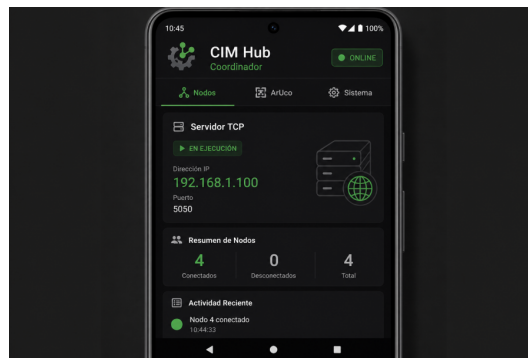


Figura: Coordinador - donde autorizo

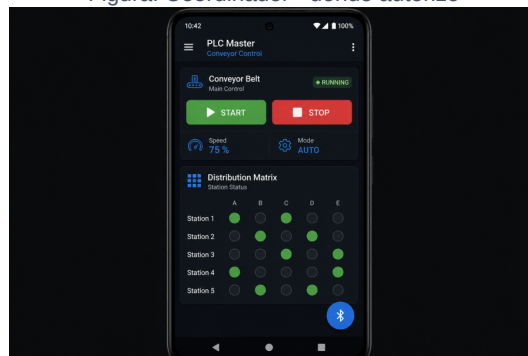


Figura: PLC - cinta



Figura: Manufactura - robot



Figura: Calidad - cámara

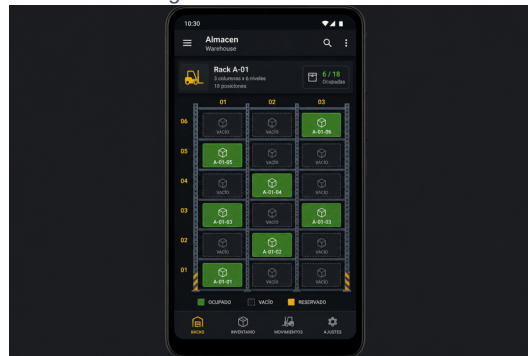


Figura: Almacén - rack

10. LO QUE NO ALCANZÓ Y POR QUÉ

- No probé Wemos real con relé con carga, porque no hay E-Stop probado. No quise arriesgar.
- YOLO no está como TFLite dentro de la APK todavía, solo como .pt afuera. Falta convertir y validar.
- BLE multiconexión con 2+ ESP32 al mismo tiempo no probado, solo simulación.
- Cámara en dispositivo real con piezas reales no probado, solo emulador.
- LAN estable con IP fija del Coordinador no probado en lab.

Esto lo dejo escrito porque es lo honesto. En la bitácora queda como pendiente de laboratorio, no como hecho.

11. CONCLUSIONES PERSONALES

Si me preguntan qué aprendí, fue que hacer un CIM no es solo código, es orden, seguridad y saber decir qué no se hizo. Al principio quería hacer que el robot se moviera al tiro, pero después entendí que eso sin E-Stop es peligroso y que la U prohíbe.

La práctica II me sirvió para aplicar ramos, pero sobre todo para aprender a dejar un repo presentable. Antes tenía 76 archivos duplicados, fotos repetidas, modelos de 19MB en 3 lados. Ahora está limpio, con 12/12 PASS y dos PDFs finales en entrega/. Eso me dejó más tranquilo que cualquier feature.

Para tesis queda pendiente lo físico: flashear cada Wemos, capturar CIM_ID por monitor serie 115200, probar sensor GPIO34, relé GPIO5 sin carga con supervisor mirando E-Stop, reconexión BLE, pallet completo. Todo con acta.

Agradezco al profe que me tuvo paciencia cuando me quedé pegado con BLE y al lab que me dejó usar las tablets. Este informe lo hice yo, contando lo que realmente pasó, sin adornos de IA.

12. BIBLIOGRAFÍA (la que realmente usé)

- Manual marca UBB (logo)
- Android Developers – CameraX, BLE
- Espressif ESP32 BLE GATT
- Scorbob manual
- YOLO docs, OpenCV ArUco
- Instructivo proyecto CIM, VALIDACION_Y_COBERTURA, SAFETY

Fin – Leonardo Araya – 2026